INGENIERIA DE SOFTWARE



MEJOR EDUCACIÓN PARA MUCHOS MÁS

Integrantes del grupo:

DI LALLA, Thiago Valentin

SCOTTO, Joaquín Alfredo

Profesor:

TORTORELLA, CHRISTIAN DAVID

UNIVERSIDAD ARGENTINA DE LA EMPRESA

FACULTAD DE INGENIERÍA Y CIENCIAS EXACTAS

Justificación de decisiones de diseño

1. Alcance funcional implementado

1.1. Funciones obligatorias

El sistema permite jugar sobre un tablero de 8 × 8, con peón, torre, caballo, alfil, reina y rey; cada pieza cuenta con sus desplazamientos permitidos. Controla la alternancia de turnos, las capturas y la legalidad de los movimientos, e identifica situaciones de jaque. Estas reglas deben existir en el núcleo y producir el mismo resultado independientemente de que la jugada provenga de la interfaz gráfica, la consola o una estrategia automática.

1.2. Funciones adicionales

También declara jaque mate, enroque, captura al paso, promoción, ahogado, triple repetición, regla de los cincuenta movimientos y material insuficiente. También declara inteligencia artificial con tres niveles, deshacer/rehacer, reloj con derrota por tiempo e interfaz gráfica configurable. Estas funciones se justifican en la medida en que permiten comprobar que la separación de responsabilidades se mantiene incluso al incorporar casos más complejos que el alcance obligatorio.

2. Separar núcleo, puertos y adaptadores

Qué decidimos. Organizamos el código en un núcleo de reglas (`core`), contratos de interacción y adaptadores externos. La lógica del ajedrez no importa Swing, procesos ni detalles de consola.

Por qué lo decidimos. La regla de un movimiento no debe depender de cómo se ingresó ni de cómo se muestra. Esto evita duplicar reglas entre interfaces y permite probar el juego sin levantar ventanas ni ejecutar Stockfish. La dirección de dependencias hacia el núcleo protege el dominio frente a cambios tecnológicos.

Cuándo la cambiaríamos o romperíamos. Si el trabajo pasara a ser un prototipo descartable con una única interfaz y sin expectativas de extensión, podría justificarse un diseño más directo. También revisaríamos el límite de los puertos si una nueva funcionalidad exige intercambios más ricos o una interfaz asíncrona.

Esta separación agrega interfaces, conversión de datos y clases de coordinación. Se acepta ese costo porque existe más de una interfaz y un motor externo, y el requisito académico exige independencia del dominio.

3. Centralizar la composición de objetos en Main

Qué decidimos. Ubicamos en `Main` la creación y conexión de implementaciones concretas, con inyección de dependencias por constructor.

Por qué lo decidimos. El punto de arranque conoce qué adaptador de entrada se utiliza y cuál es el motor disponible, mientras el núcleo consume contratos. El cambio entre consola y Swing no obliga a modificar la validación de movimientos; asimismo, un motor simulado puede sustituir al real en tests.

Cuándo la cambiaríamos o romperíamos. Si crecieran mucho las variantes de configuración o las dependencias de arranque, extraeríamos fábricas o una configuración de composición dedicada. No incorporaríamos por defecto un contenedor de inyección: solo si redujera complejidad efectiva.

4. Modelar las piezas mediante composición de comportamientos

Qué decidimos. En `core/piece` agrupamos definiciones de piezas, reglas de movimiento, rasgos, fábrica y colocación inicial. Se caracteriza explícitamente a este diseño como “piezas por composición”, en lugar de basarlo únicamente en una jerarquía profunda de subclases.

Por qué lo decidimos. Los movimientos no son idénticos: las piezas deslizantes se diferencian de los saltos del caballo y las particularidades del peón. La composición permite aislar comportamientos y reutilizar reglas donde corresponda, evitando que una superclase concentre excepciones o condicionales por tipo. También favorece introducir una pieza hipotética sin alterar todas las ya existentes.

Cuándo la cambiaríamos o romperíamos. Cambiaríamos esta decisión si la composición fragmentara injustificadamente una lógica muy pequeña o si una regla requiriera tanta coordinación entre rasgos que el modelo fuera más difícil de comprender que una implementación especializada. Una pieza nueva que necesitara reglas globales podría exigir extender la validación general, no solo crear una definición.

La composición requiere diseñar bien responsabilidades y contratos. No garantiza automáticamente apertura al cambio: una pieza con movimientos especiales puede necesitar colaboración de `core/rules`.

5. Separar movimientos candidatos de movimientos legales

Qué decidimos. Ubicamos las reglas de desplazamiento propias de cada pieza en el modelo de piezas, y en `core/rules` la comprobación de legalidad y de situaciones como jaque y reglas especiales.

Por qué lo decidimos. Una pieza puede alcanzar geométricamente una casilla, pero la jugada seguir siendo ilegal si expone a su rey. Esta distinción impide distribuir la regla global de “no dejar al rey en jaque” entre cada tipo de pieza y evita que la interfaz sea la encargada de validar.

Cuándo la cambiaríamos o romperíamos. Si se incorporaran modalidades con reglas diferentes (por ejemplo, ajedrez variante), revisaríamos si conviene parametrizar o sustituir conjuntos de reglas de validación. No cambiaríamos el modelo por una excepción visual de la GUI.

6. Representar las jugadas como comandos y mantener historial

Qué decidimos. El paquete `core/command` incluye comandos de movimiento con operaciones de ejecutar y deshacer, historial y fábrica de comandos. Se declara deshacer y rehacer tanto en la ventana como en la consola.

Por qué lo decidimos. Una jugada puede modificar más de una casilla o más de una pieza, como ocurre con el enroque, la captura al paso y la promoción. Un comando permite reunir esos efectos y los datos necesarios para revertirlos, sin hacer que la interfaz reconstruya el estado anterior. El historial aporta un punto único para navegación temporal.

Cuándo la cambiaríamos o romperíamos. Para una versión que solo permitiera avanzar, una representación de movimientos inmutables y un ejecutor sencillo podría ser suficiente. Para guardar partidas persistentes o replay distribuido evaluaríamos además eventos durables y serialización explícita; Command no resuelve por sí solo esos problemas.

7. Definir puertos de entrada específicos y observación de cambios

Qué decidimos. Distinguimos `GameActions`, `GameQueries` y `GameSubscriptions` como puertos de entrada, agrupados mediante `ChessGame`; además identifica `GameObserver` entre los puertos de salida.

Por qué lo decidimos. Esta distinción separa órdenes que cambian el estado, consultas que leen información y suscripciones a cambios. Una vista puede dibujar el tablero sin decidir si una jugada es legal, y un observador puede recibir cambios de la partida sin acoplar la lógica al mecanismo visual de Swing. Mantener contratos acotados también limita los métodos que cada consumidor necesita conocer.

Cuándo la cambiaríamos o romperíamos. Si apareciera un cliente remoto o una interfaz web, evaluaríamos contratos DTO, eventos asincrónicos o una fachada de aplicación adicional. Si un puerto se volviera demasiado pequeño y multiplicara adaptadores innecesarios, podríamos consolidar contratos sin trasladar reglas al borde.

8. Integración de Stockfish y niveles de dificultad mediante estrategias

Qué decidimos. Definimos ChessEngine como puerto de salida para integrar Stockfish 19 sin que el núcleo dependa directamente del motor externo. Además, implementamos tres niveles de dificultad: Fácil, que prioriza capturas y movimientos aleatorios; Medio, que prioriza jaques y combina Stockfish con aleatoriedad; y Difícil, que utiliza Stockfish con un Elo configurado en 1500. Si Stockfish no está disponible, se utiliza una IA propia como alternativa.

Por qué lo decidimos. Buscamos mantener el núcleo independiente de la infraestructura externa y ofrecer distintas experiencias de juego sin modificar las reglas del ajedrez.

Esta separación facilita las pruebas mediante motores simulados, permite cambiar el motor utilizado y agregar nuevas dificultades sin afectar el resto del sistema.

Cuándo la cambiaríamos o romperíamos. Si necesitáramos incorporar otros motores de ajedrez, servicios de análisis externos o nuevas dificultades, podríamos ampliar las interfaces y estrategias existentes. También revisaríamos las heurísticas y los niveles de fuerza si las pruebas demostraran que las dificultades no están correctamente equilibradas.

Los Elo (~430, ~800 y ~1500) son aproximaciones obtenidas por enfrentamientos; no constituyen certificación de ranking. El azar dificulta reproducir fallas si no se controla la semilla en tests.

9. Encapsular el estado de la partida y los turnos

Qué decidimos. El paquete `core/game` concentra partida, estados, turnos, jugadores, estrategias y reloj; `core/status` reúne condiciones de fin. La GUI consulta ese estado, pero no debe ser la autoridad de quién juega o cuándo termina una partida.

Por qué lo decidimos. Los turnos, las condiciones de jaque mate/tablas y la derrota por tiempo afectan al resultado del juego independientemente del adaptador. Situarlos en el núcleo evita incoherencias entre consola, GUI y juego contra computadora. Aislar las condiciones de fin permite razonar sobre cada causal sin fusionarla con el dibujo del tablero.

Cuándo la cambiaríamos o romperíamos. Si se incorporaran partidas multijugador en red, relojes sincronizados entre máquinas o partidas persistidas, haría falta precisar origen temporal, concurrencia y recuperación del estado. Eso podría justificar separar servicios de aplicación de las entidades puras.

10. Mantener Swing y consola como adaptadores independientes

Qué decidimos. Exponemos dos modos de ejecución sobre el mismo núcleo: `java -jar ...` para Swing y `java -jar ... --consola` para modo texto. La GUI contiene selección de configuración, temas visuales, casillas, resaltados y controles; la consola traduce comandos como `e2 e4`, `deshacer` y `rehacer`.

Por qué lo decidimos. La coexistencia de dos entradas es una prueba práctica de que la lógica no está encerrada en la vista. Permite además operar y depurar sin interfaz gráfica; cambiar colores, estilo de piezas o formato de entrada no debe modificar reglas de ajedrez.

Cuándo la cambiaríamos o romperíamos. Una interfaz web o móvil sería un tercer adaptador. Si se requiriera multijugador remoto habría que crear además comunicación de red y resolver autoridad del estado, pero el contrato del núcleo seguiría siendo un punto de partida.

11. Principios SOLID: aplicación y límites

Responsabilidad única (SRP)

Las reglas viven en `core`; representación y entrada en los adaptadores; comunicación UCI en `adapter/engine`. SRP se interpreta como motivos de cambio distintos, no como obligación de crear una clase por método.

Abierto/cerrado (OCP)

Los contratos `ChessEngine` y los comportamientos de piezas facilitan incorporar implementaciones. Sin embargo, ciertas variantes del reglamento pueden exigir modificar reglas compartidas; no es correcto afirmar apertura absoluta.

Sustitución de Liskov (LSP)

Los adaptadores que implementan un puerto deben cumplir sus expectativas: por ejemplo, una estrategia de motor no puede entregar un movimiento ilegal como si fuera válido. No hay evidencia suficiente para afirmar una jerarquía concreta sin consultar el código.

Segregación de interfaces (ISP)

La separación entre `GameActions`, `GameQueries` y `GameSubscriptions` limita dependencias de consumidores que solo necesitan mandar órdenes, consultar o escuchar cambios.

Inversión de dependencias (DIP)

El núcleo conoce puertos y no clases de Swing o Stockfish. `Main` conecta implementaciones concretas mediante constructores. La abstracción se justifica por sustitución y tests, no porque toda clase necesite interfaz.